TeamsLeech Bot

Product Requirements Document

Version 1.1 | March 2026

Live — Running on GitHub Actions · 93% Success Probability

Field	Details
Owner	Ahmed (Personal Project)
Version	1.1
Status	Live — Running on GitHub Actions
Success Probability	93%
Last Confirmed Run	March 7, 2026
Auth Strategy	OAuth2 refresh_token → access_token (auto-rotated)
Drive Discovery	Dynamic via joinedTeams API — auto-grows as new groups are joined
Subjects Monitored	6 fixed subjects
Check Modes	Per-subject button OR Check All
Infrastructure	GitHub Actions + Graph API + Telegram Bot API (Free Tier only)
Success Metric	Zero manual intervention for one full university semester

1. Overview

TeamsLeech Bot is a personal, cloud-hosted automation tool that monitors Microsoft Teams lecture recordings across 6 defined university subjects and delivers new recordings to Telegram Saved Messages — with zero local data consumption, zero cost, and zero manual intervention for a full semester.

The bot is triggered on-demand via Telegram commands. It uses the Microsoft Graph API with an OAuth2 refresh_token for authentication, runs entirely on GitHub Actions free tier, and requires no local machine to be running after the initial one-time token capture.

Drive discovery is fully dynamic: the bot calls the joinedTeams API on every check, filters teams by subject keywords, and scans all matching drives. Joining a new group mid-semester requires no configuration change — the bot finds it automatically.

Problem Statement

- Lecture recordings are locked inside Microsoft Teams / SharePoint and require the Teams app or browser to access.
- Downloading recordings consumes significant mobile data.
- Telegram provides free unlimited cloud storage — but no native Teams integration exists.
- The university IT environment blocks Azure App Registration, ruling out the official Microsoft Graph API OAuth app route.

Goal

Deliver all new lecture recordings from 6 university subjects to Telegram Saved Messages, triggered by a single bot command, using free cloud infrastructure, with no data usage on the user's device.

2. Non-Goals

The following are explicitly out of scope and will not be built, designed for, or considered during development decisions.

- Not a multi-user or SaaS tool — built for a single university account only. No login system, no user management, no shared infrastructure.
- Not for subjects outside the 6 defined ones — the bot will never check teams that do not match the 6 subject keywords, even if the user is a member of those teams.
- No real-time push or webhooks — the bot checks only when explicitly triggered. It does not listen for new uploads automatically.

- No video processing or transcription — files are streamed as-is. No compression, conversion, subtitles, or analysis.
- No local machine dependency — everything runs on GitHub Actions or Telegram after the one-time token capture.
- No admin IT cooperation required — the entire approach uses student-level Microsoft account permissions only.

---

3. Scope

In Scope

- Dynamic drive discovery via Graph API joinedTeams filtered by subject keywords
- Per-subject checking via Telegram inline buttons + Check All option
- Subject name typed as text triggers the same scan as tapping the button
- Multi-select recording download with inline Telegram checkboxes
- Direct streaming of recordings to Telegram Saved Messages (no disk writes)
- Upload progress messages posted at 10% increments, auto-deleted after success
- OAuth2 refresh_token auto-rotation via GitHub Secrets API on every run
- Session expiry detection with guided /reauth recovery flow inside Telegram

Out of Scope

- Multi-user deployment
- Azure AD app registration (blocked by university IT)
- Local machine execution during normal operation
- Real-time push notifications
- Video transcription or processing
- Subjects outside the 6 defined ones

---

4. User Stories

ID	User Story
US-01	As a student, I send /check and see 6 subject buttons + Check All to choose what to scan.
US-02	As a student, I tap a subject button and get a checklist of only new recordings for that subject.
US-03	As a student, I type a subject name (e.g. "Auditing") and get the same result as tapping its button.

US-04	As a student, I tap Check All and get a combined summary of new recordings across all 6 subjects.
US-05	As a student, I select specific recordings via checkboxes and confirm with one button.
US-06	As a student, I see upload progress per file as new messages (10% increments, no edits).
US-07	As a student, I find completed recordings in Telegram Saved Messages ready to stream.
US-08	As a student, I receive a /reauth alert with step-by-step recovery when my session expires.
US-09	As a student, new groups I join mid-semester are auto-discovered with no config change needed.

5. Auth Architecture

Microsoft blocks Azure App Registration for student accounts, so the official OAuth app route is unavailable. The solution uses the Teams Web Client's own OAuth2 flow, captured once via HAR analysis and replicated in Python.

```
TEAMS_REFRESH_TOKEN
        ↓
login.microsoftonline.com
        ↓
  access_token (~87 min)
        ↓
   Graph API calls
        ↓
Save NEW refresh_token → GitHub Secret (auto-rotated)
```

Token Lifecycle

- access_token — Generated fresh on every script run. Never stored. Valid ~87 minutes.
- refresh_token — Stored permanently as TEAMS_REFRESH_TOKEN GitHub Secret. Microsoft returns a new one on every use — the script saves it automatically.
- Manual intervention — Only needed once every ~90 days when Microsoft fully expires the refresh_token.
- Expiry handling — Bot sends alert + /reauth guided checklist inside Telegram.

Confirmed Auth Parameters

Parameter	Value

Tenant ID	7b35586a-d18d-405c-8e29-5713862937a9
Client ID	5e3ce6c0-2b1f-4285-8d4b-75ee78787346 (Teams Web Client)
Required Header	Origin: https://teams.cloud.microsoft
Token Endpoint	login.microsoftonline.com/{tenant}/oauth2/v2.0/token
Scope	https://graph.microsoft.com/.default offline_access
Recording Download	graph.microsoft.com/v1.0/drives/{id}/items/{id}/content (302 → stream)

6. Dynamic Drive Discovery

The bot never uses a hardcoded list of drive IDs. Instead, it calls the Graph API joinedTeams endpoint on every check run and filters teams by subject keywords defined in subjects_config.json. New groups joined mid-semester are automatically included.

Subject Keyword Mapping

Subject	Keywords (case-insensitive)	Drive Count
Advanced Database	advanced database, advanced db, adv db, adv database	11 teams confirmed
Auditing	audit, auditing	4+ drives
Economics of Information	economics of information, econ of info	2+ drives
Internet Applications	internet applications, internet app	5+ drives
Management Information Systems	management information system, mis	8+ drives
Operations Research	operation research, o.r, o.r.	6+ drives

- Confirmed live: 98 total joined teams fetched per run
- Keywords stored in subjects_config.json — extend without changing Python code
- Matching is case-insensitive on team displayName
- joinedTeams API pagination handled automatically — supports 999+ teams

7. Check Flow

Step / Trigger	What Happens

User sends /check	Bot presents 6 subject buttons + Check All button
User taps a subject	Bot scans only drives whose team name matches subject keyword
User types subject name	Same result as tapping the button — text matching supported
Fetcher scans matching drives	Finds .mp4 files newer than last_run.txt timestamp
Bot sends subject summary	Inline checkboxes listing new recordings with name, size, date
User selects and confirms	Selected files stream to Telegram Saved Messages
User taps Check All	All 6 subjects scanned sequentially, combined summary returned

Telegram UI

```
What do you want to check?

[ Advanced DB ]  [ Auditing ]  [ Econ of Info ]
[ Internet Apps ]  [ MIS ]  [ O.R. ]
[ Check All ]
```

---

8. Feature Specifications

F1 — /check + Subject Button Interface

- Responds with 6 subject inline keyboard buttons + Check All
- Subject button: scans only drives matching that subject's keywords via live joinedTeams call
- Check All: scans all 6 subjects sequentially, returns combined summary
- Subject name typed as text triggers the same scan as the button
- No new recordings: "No new recordings found for [Subject]"

F2 — Multi-Select Upload

- Results grouped by subject with inline checkboxes per recording
- Each item shows: recording name, size in MB, creation date, team name
- User selects any combination, confirms with Upload Selected button

F3 — Upload Progress

- New Telegram message posted per file at: 10%, 20%, 30% ... 100%
- Never edits existing messages — avoids Telegram rate limits
- Progress messages auto-deleted after successful upload — chat stays clean
- Final: filename — size MB — Saved to Telegram

F4 — /reauth Guided Recovery

- Triggered when refresh_token is fully expired (~90 days)
- Bot sends: Session expired. Use /reauth to recover.
- /reauth sends 4-step Telegram checklist — purely instructional, no server needed
- Step 1: Open Teams in browser → F12 → Network tab → login
- Step 2: Find login.microsoftonline.com POST → copy refresh_token from request body
- Step 3: GitHub repo → Settings → Secrets → Update TEAMS_REFRESH_TOKEN
- Step 4: Send /check to verify

F5 — Token Auto-Rotation

- Every successful auth call returns a new refresh_token from Microsoft
- token_manager.py encrypts it using PyNaCl + repo public key
- Saves updated token to TEAMS_REFRESH_TOKEN GitHub Secret automatically
- On save failure: Telegram alert sent before script exits

---

9. System Components

Component	Responsibility	Phase
token_manager.py	refresh_token → access_token + auto-rotate via GitHub Secrets API	Phase 1
fetcher.py	joinedTeams scan → keyword filter → .mp4 search → date filter → results	Phase 2
bot.py	/check buttons, /reauth, inline keyboard, text command matching	Phase 3
uploader.py	Stream /content → Telegram + 10% progress messages (auto-deleted on success)	Phase 4
main.py	Orchestrator — reads GitHub Secrets env vars, calls all modules	Phase 5
workflow.yml	GitHub Actions workflow_dispatch trigger + all secrets as env vars	Phase 6
subjects_config.json	6 subjects → keyword arrays (editable without code changes)	Config

---

10. Tech Stack

Layer	Technology
Language	Python 3.11
Auth	OAuth2 refresh_token flow via requests (no browser automation)
Token Encryption	PyNaCl (libsodium) for GitHub Secrets API writes
API Client	requests — Bearer token injection on every call
Telegram Client	Pyrogram + TgCrypto
CI/CD Runner	GitHub Actions ubuntu-latest (manual trigger only)
Secrets Management	GitHub Encrypted Secrets
Drive Discovery	Live Graph API joinedTeams + subjects_config.json keywords
State Persistence	GitHub Actions Artifacts (last_run.txt per subject)
Video Streaming	Graph API /content endpoint, temp file + send_video() + supports_streaming=True

11. GitHub Secrets Required

Secret Name	Description
TEAMS_REFRESH_TOKEN	OAuth2 refresh token — captured once via HAR, auto-rotated every run
GH_PAT	Personal Access Token with secrets:write permission on this repo
TELEGRAM_API_ID	From my.telegram.org
TELEGRAM_API_HASH	From my.telegram.org
TELEGRAM_BOT_TOKEN	From @BotFather
TELEGRAM_CHAT_ID	Your Saved Messages chat ID
TELEGRAM_SESSION	Pyrogram session string — generated once, never expires

12. Build Phases

Phase	Name	Key Deliverable	Status	Confidence
Phase 0	Token Capture + Config	refresh_token captured, subjects_config.json written	☑ Done	High
Phase 1	token_manager.py	Token rotation confirmed on GitHub Actions	☑ Done	High

Phase 2	fetcher.py	98 teams fetched, keyword filter working	☑ Done	High
Phase 3	bot.py	/check, subject buttons, checkboxes, rename working	☑ Done	High
Phase 4	uploader.py	148MB and 238MB uploaded successfully	☑ Done	High
Phase 5	main.py	Orchestrator running on GitHub Actions	☑ Done	High
Phase 6	workflow.yml	workflow_dispatch confirmed green	☑ Done	High
Phase 7	End-to-End Test	Live run completed March 7, 2026	☑ Done	High

13. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
refresh_token expires before 90-day reminder	Low	High	Calendar reminder at day 75. Auto-rotation confirmed working — rotates on every run.
Script fails to save rotated refresh_token	Very Low	High	try/catch in token_manager — Telegram alert on failure before exit.
Keyword mismatch misses a new subject group	Medium	Medium	subjects_config.json supports multiple keywords per subject — easy to extend.
joinedTeams API pagination truncates results	Low	Medium	@odata.nextLink pagination implemented — handles 999+ teams.
Telegram 2GB file size limit exceeded	Low	Medium	Auto-split files > 2GB into parts. Most lectures are 150–450MB.
Microsoft revokes Teams Web Client app token	Low	High	Requires fresh HAR capture + /reauth flow. No automated mitigation possible.
BytesIO RAM failure on GitHub Actions	Low	High	Resolved — temp file approach implemented.
GitHub Actions free minutes exhausted	Low	Low	Confirmed: ~8 min/run, ~200 min/month vs 2000 free.

14. Success Criteria

The project is complete and successful when all of the following are true:

- ☑ Bot replies to /check with 6 subject buttons within 5 seconds
- ☑ Per-subject scan correctly finds all drives for that subject dynamically
- ☑ New groups joined mid-semester are discovered automatically with no config change
- ☑ Selected recordings stream to Telegram Saved Messages with zero local data usage
- Progress messages appear at every 10% increment without Telegram rate-limiting
- ☑ refresh_token rotates silently on every run — zero manual intervention required
- System runs without manual intervention for one full university semester
- Session expiry triggers /reauth guide resolving in under 5 minutes
- Total infrastructure cost remains $0 throughout the entire semester

---

15. Production Notes

Metric	Confirmed Value
Teams fetched per run	98
Advanced DB teams matched	11
Recordings found (first run)	14
Download speed (GitHub Actions)	~148MB in 5s, ~238MB in 7s
Upload method	Temp file → send_video() + supports_streaming=True
Token rotation	Confirmed working — GitHub Secret updated each run
Known log warnings	Unclosed tag (cosmetic), MessageNotModified (cosmetic)
Artifact error on first run	Expected — continue-on-error handles it
Run duration	~8 minutes for full session

End of Document — TeamsLeech Bot PRD v1.1 — March 2026